1. Diamond rush

Parent item	Proyectos SisInt
Created time	12 de septiembre de 2026 21:51
Last modified	25 de septiembre de 2026 12:30
Tipo	

▼

Primer proyecto → 4 semanas → 5 de octubre

- Qué?
  - Agente inteligente capaz de jugar diamond rush
    - Completando la mayor cantidad de niveles lo más rapido
    - Puede empezar en cualquier nivel
    - Debe recibir la imagen e interpretarla y jugarla
    - Tiene que procesarse desde la interfaz gráfica, no se puede descargar
- Qué no se puede hacer
  - Web scrapping
  - Cualquier lenguaje se puede usar
  - Se puede traer su propio compu, pero reconocer colores no es tarea trivial, cambia entre compu y compu
- torneo
  - cada grupo con su compu
  - El que gane el torneo lo presenta asi que ojo con el vibe coding
  - Cantidad de niveles y tiempo

Mi propuesta pre IA

Lenguaje: python

Plataforma: Vscodium

Cómo idea general, la clase lo apuntaba a uno a usar Heuristicas, es decir cómo lo entiendo yo, generalizar el camino mas corto a nuestra meta, subestimando el costo, para que sea valido, es decir para poder hacer el calculo del costo ponderado, que nos sirve para comparar cada ruta.

Bien

1. Averiguar cómo hacer para que se pueda identificar sin scrapping, los elementos del juego y el camino, mi naive idea, es usar un modelo y entrenarlo en muchos niveles para que identifique cuales son puestos caminables, cuales son diaments, y en general los elementos del juego, tiene que identificar todos 100% bien
2. Luego implementar search_route_bfs, esto es con una cola prioritaria supongo, haciendo el analisis con heuristicas, que entiendo que es como funciona el algoritmo A*, por ahora dado que es una heuristica se ignorarían los obstaculos, y solo seria un BFS entre diamante y diamante hasta el ultimo diamante de donde se va para la entrada es decir: del scan ia se obtien cant_total_diamantes, cuando esto == diamantes_colectados, se cambia el bfs hacia la salida(todo esto marcando las paredes como areas inaccesibles), n ose com ose implementa esto en particular, me falta estudiarle a DSA, o bueno practicar, siempre es un mapa de 10×15, entonces seria 2 for, uno anidado, para escanear supongo..., y el for de filas identificaria cuales casillas son cuales, luego con los indices de la matriz se crearia la ponderación, cada casilla es peso 1, o bueno cada movimiento, es decir el valor absoluto de un movimiento, en el plano xy.
3. Luego... uff esto es l odificil, los constraints:
    a. Minimo peso/camino
    b. los obstaculos
        i. Pinchos: elemento que al pasarlo se convierte en bloque no pasable [seria X, en vez de O]
        ii. Llave y puertas de llave es como si hubiera un sub mapa en el mapa 10×15 pienso yo porque hay que priorizar la llave, luego de eso el mapa cambia y la prioridad es la puerta, pero debe ser

dinamicamente programado, para entrehilarlo con diamantes y otros obstaculos ,esto es lo complicado, porque hay estados del juego que bloquean el nivel si se hacen mal, no se si esos se pueden identificar a fuerza bruta o generalizar, puede ser que sea una condicion de door_is_locked y has_key + there_is_path(esto pq si los pinchos o las rocas lo bloquean paila) y si uno de esos tienen banderas especificas, tipo no hay camino pa la puerta, simplemente no se puede seguir.

iii. Rocas de empujar: similar pero diferente a lo anterior, las rocas dependen de: posición, prioridades, es decir, si luego de usar una roca, no se puede acceder a la llave, mejor dicho, la clave es tener el is_path muy claro, quizas se puede crear un arbol dirigido, estoy viendo que este proyecto es muy de dinamicamente entrelazar jerarquias de prioridades, mas lectura de mapa, mas bfs.

iv. Botones y puertas de botones: objetos que presionan: rocas y personaje, los huecos y los botones pueden tratarse similar, pero seria un hibrido entre como se tratan a las llaves y a los huecos, quizas una clase Openable?

4. En teoria despues de constraints, y de que el mansito sea capaz de completar niveles se trata de optimizar esta monda, optimizar grafos, usar hessianos. etc

Estrategia corregida con IA

Notas

- Entrenamiento no, usar template matching, con OpenCV cortando los sprites fijos
- En efecto se de be usar A* para hacer busqueda en espacio de estados(espacio de busqueda), con una tupla que no evalue posicion solamente, sino que pos_jugador, estado del mapa e invetario.
- Los hesianos valen monda, pues es un problema discreto entonces gradient descent no funciona, la optimización de be enfocarse en la función heuristica h(n) de A*, algo de distancia manhattan.
- Si se usa RGB, los rangos entre portatiles romperan el sistema, entonces implementar un modulo de calibración inicial, para normalizar la imagen antes de procesarla

    - Esta quizas no funcione, debo de preguntar al profesor, pues esto puede no estar permitido en las reglas del proyecto. ademas el tamaño de pantalla y nivel siempre será el mismo.

Estructura del agente

- Performance measure
    - Level completion
    - amount of tiles
    - Level being blocked
    - Penalizacipon por reinicios
    - Cnatidad de diamantes recolectados
- Environment
    - The 10×15 level
    - Obstacles
- Actuators
    - The movements, up, down, left, right, reset
- Sensors
    - Capturador de pantalla en tiempo real

Fases del proyecto

1. Captura de pantalla y procesamiento de imagen
    a. mss, pillow(capturar pantalla en python)
    b. Opencv, cv2.matchTemplate, para comparar los sprites con el nivel
    c. NumPy para representar y manipular la matriz
2. Modelo del entorno y reglas del juego
    a. Estructura de datos que define el espacio de busqueda
    b. usar python nativo xd
3. Algoritmo de busqueda y planificación

a. Secuencia optima de pasos

b. Heapq, para A*

c. Heuristica con jerarquia, distancia manhattan considerando constraints(Llave → PUerta → Roca → boton → diamantes → salida)

4. Ejecucion de acciones y sincronización

a. Enviar teclas

b. pydirectinput o pyautogui

c. time para delays entre pasos

Propuesta con IA

Propuesta de Proyecto: Agente Inteligente para Diamond Rush

Fecha límite: 5 de Octubre (4 semanas)

Lenguaje principal: Python 3.x

Entorno de desarrollo: VSCodium / VS Code

1. Alcance y Objetivo del Proyecto

Desarrollar un agente autónomo basado en búsquedas heurísticas capaz de interpretar visualmente el juego *Diamond Rush* desde la interfaz gráfica (sin escrapear código ni acceder a la memoria), planificar la secuencia óptima de movimientos y resolver los niveles en el menor tiempo posible.

2. Arquitectura del Agente (PEAS)

- **Rendimiento (P):** Porcentaje de niveles resueltos, tiempo total de ejecución y recolección máxima de diamantes.

- **Entorno (E):** Tablero discreto de 10×15, estático por turnos y determinista.
- **Actuadores (A):** Emulación de eventos de teclado (flechas de dirección).
- **Sensores (S):** Captura de pantalla en tiempo real (frame buffer / ROI).

3. Estrategia de Solución Téscnica

```
 +------------------+     +--------------------+     +-------------------+     +------------+
 | Captura y Visión | --> | Matriz de Estado   | --> | Motor de Búsqueda | --> | Emulador Teclado |
 | (mss + OpenCV)   |     | (NumPy Array 10x15)|     | (A* Search)       |     | (PyDirectInput)  |
 +------------------+     +--------------------+     +-------------------+     +------------+
```

1. **Visión por Computador (Sin Machine Learning):**
   - Usaremos **Template Matching** (`OpenCV`) sobre una región de interés calibrada.
   - La pantalla se dividirá en una rejilla de 10×15 casillas y se comparará cada celda contra una librería de sprites estáticos (diamantes, rocas, puertas, llaves).
   - **Ventaja:** Cero tiempo de entrenamiento, 100% de precisión y ejecutable en cualquier portátil.
2. **Búsqueda y Planificación (A* sobre Espacio de Estados):**
   - Un simple algoritmo BFS de posiciones no funciona debido a las dinámicas del juego (empujar rocas, recoger llaves, activar pinchos).
   - Implementaremos el algoritmo **A*** donde el estado no es solo (x, y), sino la combinación de: `(Posición_Jugador, Estado_Matriz_Tablero, Objetos_Inventario)`.
   - La función heurística $h(n)$ evaluará la distancia Manhattan hacia el objetivo activo con mayor prioridad (ej. Llave $\rightarrow$ Puerta $\rightarrow$ Diamantes $\rightarrow$ Salida).
3. **Ejecución y Control:**

- Una vez calculada la ruta de estados, el agente traducirá la lista de nodos en una secuencia de teclas con *delays* mínimos para coincidir con la velocidad de animación del juego.

4. Plan de Trabajo por Semanas

Semana	Módulo	Entregables clave
Semana 1	**Visión y Lectura**	Calibración de pantalla + Mapeo correcto de la matriz 10×15 usando OpenCV.
Semana 2	**Lógica del Juego**	Definición de reglas de colisión, empuje de objetos, llaves/puertas y estados imposibles (*softlocks*).
Semana 3	**Motor de Búsqueda A***	Implementación de A* sobre el espacio de estados y ajuste de la función heurística.
Semana 4	**Pruebas y Torneo**	Optimización de tiempos de ejecución, calibración de portátiles y pruebas en tiempo real.

Misc

- Espacio 10 × 15
- 19 niveles

- Diamantes
- Puerta
- Pinchos
- Llave
- Puertas de llave
- Rocas de empujar
- Agujeros
- Botones

- Puertas de botones

Niveles

- Nivel 12
  - 7 puas
  - 8 diamantes
  - 3 rocas
  - 1 agujero
  - 1 par llave/puerta
  - 1 par botón/puerta